Планирование процессов

Тема 5

Уровни планирования

Долгосрочное планирование — это уровень планирования, который решает, какие новые процессы запускать в системе. Оно определяет степень мультипрограммирования (сколько процессов одновременно находится в системе) и выполняется сравнительно редко.

Среднесрочное планирование — это уровень, который управляет временным выгружением процессов из оперативной памяти на диск (свопингом) и их возвратом обратно. Его задача — оптимизировать загрузку памяти и повысить производительность системы.

Краткосрочное планирование — это уровень, который выбирает из готовых процессов тот, который получит процессор и будет выполняться следующим. Оно выполняется очень часто — обычно каждые десятки или сотни миллисекунд.

Критерии планирования и требования к алгоритмам

- **Справедливость** – гарантировать каждому заданию или процессу определенную часть времени использования процессора в компьютерной системе, стараясь не допустить возникновения ситуации, когда процесс одного пользователя постоянно занимает процессор, в то время как процесс другого пользователя фактически не начинал выполняться.
- **Эффективность** – постараться занять процессор на все 100% рабочего времени, не позволяя ему простаивать в ожидании процессов, готовых к исполнению. В реальных вычислительных системах загрузка процессора колеблется от 40 до 90%.
- **Сокращение полного времени выполнения** (turnaround time) – обеспечить минимальное время между стартом процесса или постановкой задания в очередь для загрузки и его завершением.
- **Сокращение времени ожидания** (waiting time) – сократить время, которое проводят процессы в состоянии готовность и задания в очереди для загрузки.
- **Сокращение времени отклика** (response time) – минимизировать время, которое требуется процессу в интерактивных системах для ответа на запрос пользователя.

Свойства алгоритмов планирования

- Предсказуемость
- Минимальные накладные расходы
- Равномерная загрузка ресурсов вычислительной системы
- Масштабируемость

Параметры планирования

Статические параметры не изменяются в ходе функционирования вычислительной системы.

Динамические же, напротив, подвержены постоянным изменениям.

Динамические параметры

Деятельность любого процесса можно представить как последовательность циклов использования процессора и ожидания завершения операций ввода-вывода. Промежуток времени непрерывного использования процессора носит название CPU burst, а промежуток времени непрерывного ожидания ввода-вывода – I/O burst. На рисунке 1. показан фрагмент деятельности некоторого процесса на псевдоязыке программирования с выделением указанных промежутков.

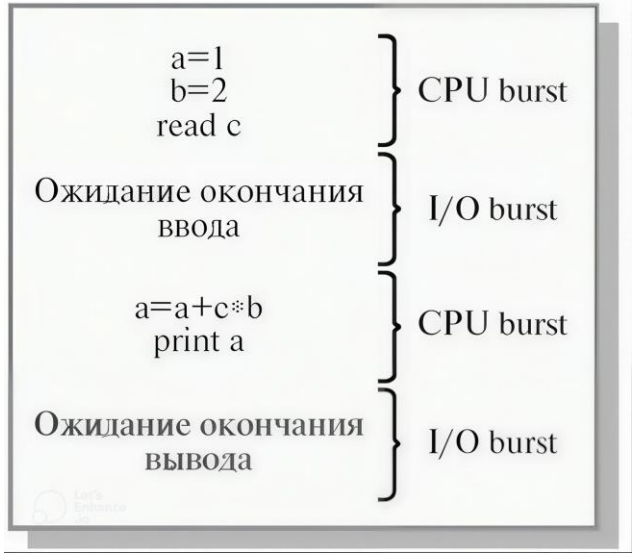

Рис.1 Фрагмент деятельности процесса с выделением промежутков непрерывного использования процессора и ожидания ввода-вывода

Вытесняющее и невытесняющее планирование

Планировщик может принимать решения о выборе для исполнения нового процесса из числа находящихся в состоянии готовность в следующих четырех случаях:

1. Когда процесс переводится из состояния исполнение в состояние закончил исполнение.
2. Когда процесс переводится из состояния исполнение в состояние ожидание.
3. Когда процесс переводится из состояния исполнение в состояние готовность (например, после прерывания от таймера).
4. Когда процесс переводится из состояния ожидание в состояние готовность (завершилась операция ввода-вывода или произошло другое событие).

Алгоритмы планирования

First-Come, First-Served (FCFS) - простейшим алгоритмом планирования является алгоритм, который принято обозначать аббревиатурой FCFS по первым буквам его английского названия – First-Come, First-Served (первым пришел, первым обслужен).

Таблица 3.1.

Процесс	p_0	p_1	p_2
Продолжительность очередного CPU burst	13	4	1

Рис.2 Выполнение процессов при порядке p0,p1,p2

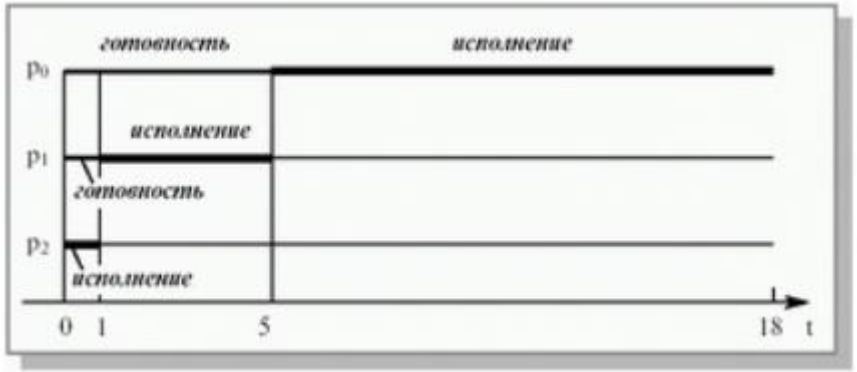

Рис.3 Выполнение процессов при порядке p2, p1, p0

Алгоритмы планирования

Round Robin - модификация алгоритма FCFS. По сути дела, это тот же самый алгоритм, только реализованный в режиме вытесняющего планирования. Можно представить себе все множество готовых процессов организованным циклически – процессы сидят на карусели. Карусель вращается так, что каждый процесс находится около процессора небольшой фиксированный квант времени, обычно 10 – 100 миллисекунд (см. рис. 4.). Пока процесс находится рядом с процессором, он получает процессор в свое распоряжение и может исполняться.

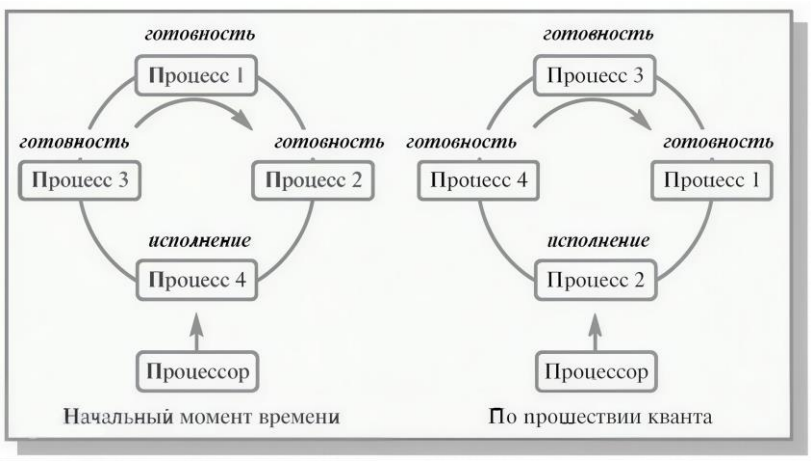

Рис.4 Процессы на карусели

Таблица 3.2.

Время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
p_0	И	И	И	И	Г	Г	Г	Г	Г	И	И	И	И	И	И	И	И	И
p_1	Г	Г	Г	Г	И	И	И	И										
p_2	Г	Г	Г	Г	Г	Г	Г	Г	И									

Таблица 3.3.

Время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
p_0	И	Г	Г	И	Г	И	Г	И	Г	И	И	И	И	И	И	И	И	И
p_1	Г	И	Г	Г	И	Г	И	Г	И									
p_2	Г	Г	И															

Алгоритмы планирования

Shortest-Job-First (SJF) - алгоритм краткосрочного планирования может быть как вытесняющим, так и невытесняющим.
- При невытесняющем SJF-планировании процессор предоставляется избранному процессу на все необходимое ему время, независимо от событий, происходящих в вычислительной системе.
- При вытесняющем SJFпланировании учитывается появление новых процессов в очереди готовых к исполнению (из числа вновь родившихся или разблокированных) во время работы выбранного процесса.
- Если CPU burst нового процесса меньше, чем остаток CPU burst у исполняющегося, то исполняющийся процесс вытесняется новым

Таблица 3.4.

Процесс	p_0	p_1	p_2	p_3
Продолжительность очередного CPU burst	5	3	7	1

Таблица 3.5.

Время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
p_0	Г	Г	Г	Г	И	И	И	И	И							
p_1	Г	И	И	И												
p_2	Г	Г	Г	Г	Г	Г	Г	Г	Г	И	И	И	И	И	И	И
p_3	И															

Shortest-Job-First (SJF)

Таблица 3.6.

Процесс	Время появления в очереди очередного CPU burst	Продолжительность
p_0	0	6
p_1	2	2
p_2	6	7
p_3	0	5

Таблица 3.7.

Время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
p_0	Г	Г	Г	Г	Г	Г	Г	И	И	И	И	И	И							
p_1			И	И																
p_2							Г	Г	Г	Г	Г	Г	Г	И	И	И	И	И	И	И
p_3	И	И	Г	Г	И	И	И													

Гарантированное планирование

Для каждого пользователя учитываются две величины:
Ti — сколько времени он уже находится в системе (длительность его сеанса);
τi — сколько процессорного времени система уже дала всем его процессам за этот сеанс.

Если бы всё было идеально справедливо, то пользователь за время Ti должен был бы получить примерно **Ti / N** процессорного времени.
Дальше сравниваем:
Если **τi намного меньше Ti / N**, значит пользователь получил меньше, чем «положено» — система его обделяет.
Если **τi намного больше Ti / N**, значит он уже получил больше своей доли.
Чтобы автоматически поддерживать баланс, вводят коэффициент справедливости:

$$\tau_i N / T_i$$

Алгоритм работает так:
процессор получает тот готовый процесс, у которого этот коэффициент **минимальный**.
То есть система в первую очередь даёт время тому, кто получил меньше всего относительно своей доли.

Приоритетное планирование

Приоритетное планирование — это универсальная дисциплина обслуживания, при которой выбор процесса из очереди на исполнение осуществляется на основе присвоенного ему числового значения (**приоритета**).

Таблица 3.8.

Процесс	Время появления в очереди	Продолжительность очередного CPU burst	Приоритет
p_0	0	6	4
p_1	2	2	3
p_2	6	7	2
p_3	0	5	1

Таблица 3.9.

Время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
p_0	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	И	И	И	И	И	И
p_1			Г	Г	Г	И	И													
p_2							Г	И	И	И	И	И	И	И						
p_3	И	И	И	И	И															

Таблица 3.10.

Время	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
p_0	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	И	И	И	И	И	И
p_1			Г	Г	Г	И	Г	Г	Г	Г	Г	Г	Г	И						
p_2							И	И	И	И	И	И	И							
p_3	И	И	И	И	И															

Многоуровневые очереди (Multilevel Queue)

Для систем, в которых процессы могут быть легко рассортированы по разным группам, был разработан другой класс алгоритмов планирования. Для каждой группы процессов создается своя очередь процессов, находящихся в состоянии готовность (см. рис.5). Этим очередям приписываются фиксированные приоритеты.

Рис.5 Несколько очередей планирования

Многоуровневые очереди с обратной связью (Multilevel Feedback Queue)

Дальнейшим развитием алгоритма многоуровневых очередей является добавление к нему механизма обратной связи. Здесь процесс не постоянно приписан к определенной очереди, а может мигрировать из одной очереди в другую в зависимости от своего поведения.

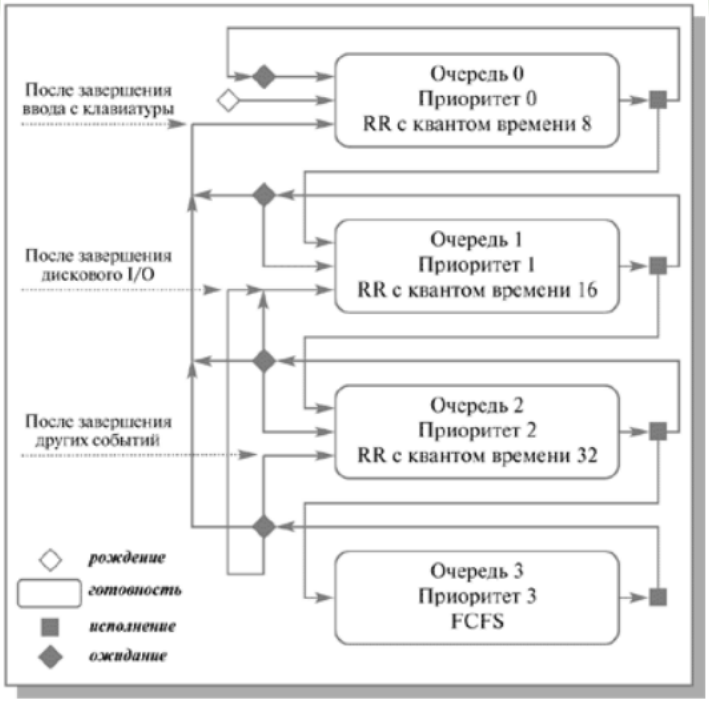

Рис.6 Схема миграции процессов

Заключение

- **Процессорное время** — один из самых ценных и ограниченных ресурсов вычислительной системы. Поскольку одновременно в системе работает множество процессов, необходимо грамотно распределять между ними этот ресурс. Эту задачу решает планирование процессов.
- **Существует три уровня планирования**: долгосрочное, среднесрочное и краткосрочное. Они отличаются тем, как долго их решения влияют на работу системы — от редких стратегических решений до частых и оперативных.
- **Алгоритмы планирования** выбираются в зависимости от целей системы: нужно ли минимизировать время ожидания, обеспечить справедливость между пользователями или повысить отзывчивость. При этом используются как постоянные характеристики процессов, так и динамически изменяющиеся параметры.
- **Планирование** может быть невытесняющим, когда процесс сам освобождает процессор, или вытесняющим, когда система может принудительно передать процессор другому процессу.
- **Существуют разные алгоритмы планирования**. Простейший — FCFS — прост в реализации, но может быть неэффективным для коротких задач. Более гибкие алгоритмы, такие как RR, лучше подходят для систем разделения времени. Алгоритм SJF минимизирует среднее время ожидания, а гарантированное планирование обеспечивает справедливое распределение процессорного времени между пользователями. Более сложные приоритетные схемы с многоуровневыми очередями позволяют системе адаптироваться к различным типам задач.
- **В целом**, чем сложнее алгоритм планирования, тем более гибкой и адаптивной становится система, но тем выше требования к её реализации.